

**Natural Language
Processing (NLP) Project**

"Sentiment Analysis System"

Team Members:

Raghad Alharbi – 444011371

Maria Alsharif – 444009824

Ghala Allahibi – 444009399

Ruyuf Aljubayri – 444008745

Supervised

Dr. Amal Alshahrani

Introduction

Sentiment Analysis is a branch of Natural Language Processing (NLP) that focuses on identifying and analyzing the opinions and emotions expressed in written texts. It aims to determine whether a piece of text conveys a positive, negative, or neutral sentiment.

This technique is highly important for automatically understanding customer and user feedback, such as product reviews, website comments, and social media posts like tweets

With the rapid growth of online platforms, large volumes of textual data are generated every day, making manual analysis inefficient and time-consuming. Sentiment analysis provides an effective solution by enabling organizations to quickly analyze user opinions, monitor public sentiment, and make data-driven decisions.

As a result, sentiment analysis has become widely used in areas such as marketing analysis, customer service improvement, and opinion mining

Project Objective

The objective of this project is to develop a system capable of analyzing sentiments in written texts, enabling it to classify texts into three categories: **Positive**, **Negative**, or **Neutral**.

This system is applied to various types of text data, such as product reviews, user comments, or tweets, in order to automatically and accurately extract users' opinions and emotions

Project Requirements

Functional Requirements (FR)

1. **FR1:** Accept a single text or multiple texts from the user or a dataset.
2. **FR2:** Perform text preprocessing (cleaning, removing unnecessary symbols, tokenization).
3. **FR3:** Convert texts into a numerical representation suitable for machine learning, such as TF-IDF or embeddings.
4. **FR4:** Apply a sentiment classification model (Positive / Negative / Neutral).
5. **FR5:** Display the classification result to the user with a confidence score.
6. **FR6:** Save the trained model for later use

Non-Functional Requirements (NFR)

1. **NFR1:** The system should achieve an acceptable accuracy ($\geq 75\%$)
2. **NFR2:** The system should operate efficiently and return results within 2 seconds per text
3. **NFR3:** The user interface should be simple, clear, and easy to use
4. **NFR4:** The system should be scalable to add more data or models easily
5. **NFR5:** The code should be organized, readable, and maintainable

Dataset Requirements (DR)

1. Provide a dataset containing texts pre-labeled as Positive, Negative, or Neutral
2. Clean the dataset by removing duplicates, empty texts, or irrelevant entries
3. Split the dataset into training (Train) and testing (Test) sets
4. Save the dataset in a suitable format such as CSV or JSON for easy processing during training and use

Corpus / Dataset Collection

For this project, the **Amazon Fine Food Reviews Dataset** was used
This dataset is publicly available on Kaggle and contains customer reviews of food products sold on Amazon

Dataset Source

- Platform: Kaggle
- Dataset Name: *Amazon Fine Food Reviews*
- Link: <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews>
-

Dataset Description

The dataset consists of textual reviews written by users, along with numerical ratings that reflect customer satisfaction with the product. Each review includes the review text and a score ranging from 1 to 5

- Reviews with high scores represent **positive sentiment**
- Reviews with low scores represent **negative sentiment**
- Medium scores can be treated as **neutral sentiment**

Dataset Size

- Total number of reviews: **568,454 reviews**
- Language: English
- Data format: CSV

Sentiment Labels

For sentiment analysis, the ratings were mapped into sentiment classes as follows:

- **Positive:** Score = 4 or 5
- **Neutral:** Score = 3
- **Negative:** Score = 1 or 2

Preprocessing Operations

The preprocessing stage included several steps applied to both textual and numerical data, as follows:

Data Loading and Reduction

Only the first **61,946** rows were loaded from the dataset file (*Reviews_sample.csv*) using the `nrows` parameter in order to reduce memory usage during processing. Rows containing missing values in the **Text** or **Score** fields were removed

Label Mapping (Score Mapping)

The original numerical scores (ranging from 1 to 5) were mapped into three sentiment classes:

- **0:** Negative
- **1:** Neutral
- **2:** Positive
-

Text Cleaning

The following text cleaning steps were applied:

- **Lowercasing:** All text was converted to lowercase to ensure consistency.
- **Removing unwanted elements:** URLs (http/https), non-alphabetic characters, and punctuation marks were removed
- **Stopword removal:** Common words that do not contribute to sentiment meaning (such as “the”, “is”, “a”) were removed using the `nltk.corpus.stopwords` list
- **Whitespace normalization:** Extra whitespace was removed

TF-IDF Vectorization

The cleaned text was transformed into numerical features using **TF-IDF (Term Frequency–Inverse Document Frequency)**, allowing the machine learning model to process the textual data

- The maximum number of features was set to **10,000**.
- The parameter `ngram_range = (1, 2)` was used to include both unigrams and bigrams in the analysis

Data Splitting

The cleaned and vectorized data was split into training and testing sets using the `train_test_split` function as follows:

- **Split ratio**
 - 80% for training (X_{train}, y_{train})
 - 20% for testing (X_{test}, y_{test})
- **Reproducibility**

The parameter `random_state = 42` was used to ensure consistent and reproducible results across different runs
- **Stratification**

The parameter `stratify = y` was applied to preserve the original class distribution (positive, negative, neutral) in both the training and testing sets. This helps prevent bias and ensures fair evaluation
- **TF-IDF application**

The TF-IDF transformer was fitted only on the training data using `fit_transform`, and then applied to the test data using `transform` to avoid data leakage

Model Training

After completing the preprocessing and feature extraction steps, a machine learning model was trained to perform sentiment classification on the prepared dataset. The dataset was represented using TF-IDF feature vectors generated from the cleaned text data

A **Logistic Regression** classifier was selected for this task due to its simplicity, efficiency, and strong performance in text classification problems. The model was trained using the TF-IDF features extracted from the training set (**X_{train}**) and their corresponding sentiment labels (**y_{train}**)

During the training process, the model learned the relationship between textual features and sentiment classes (negative, neutral, and positive). The trained model was then prepared for evaluation using the unseen test dataset to assess its performance and generalization ability

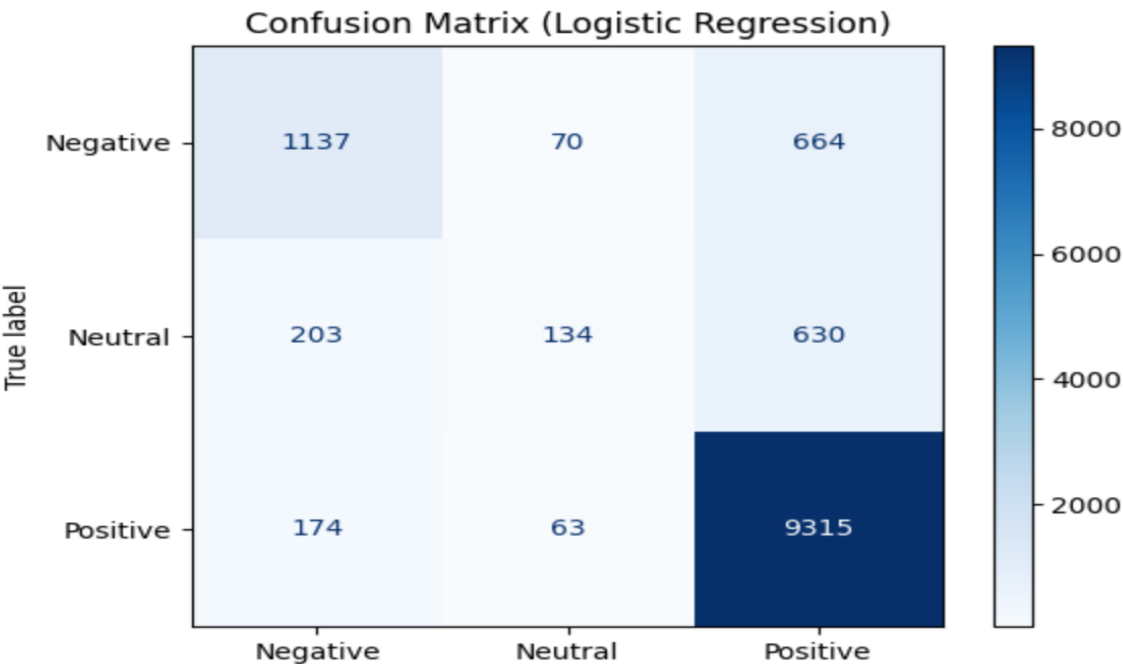
Evaluation Results

The model shows good overall performance with an accuracy of around 85% it performs very well in identifying positive sentiment, correctly classifying most positive reviews. The negative sentiment class achieves acceptable results, while the neutral class is more difficult to classify due to class imbalance and the ambiguous nature of neutral reviews overall, the results indicate that using TF-IDF with logistic regression is an effective method for sentiment analysis

--- Step 8: Evaluation Results ---
Accuracy: 0.8544

Classification Report:

	precision	recall	f1-score	support
Negative (0)	0.75	0.61	0.67	1871
Neutral (1)	0.50	0.14	0.22	967
Positive (2)	0.88	0.98	0.92	9552
accuracy			0.85	12390
macro avg	0.71	0.57	0.60	12390
weighted avg	0.83	0.85	0.83	12390



Testing

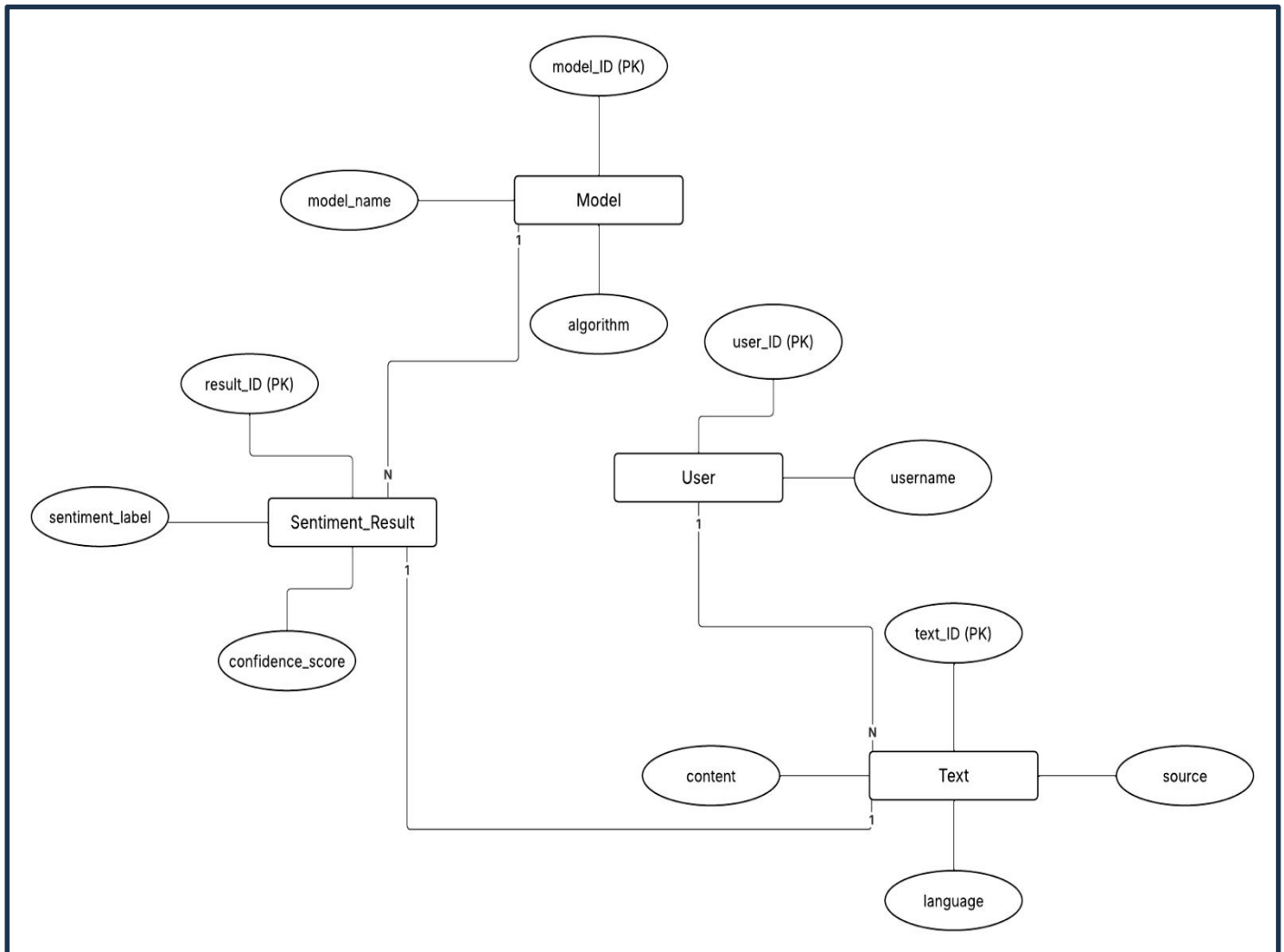
--- Step 9: Live Prediction on New Sentences ---

	Original Text	Prediction
0	I don't hate this product at all! Its quality ...	Negative (سلبی)
1	The delivery was late, but the item itself is ...	Neutral (محايد)
2	This service is incredibly slow and disappoint...	Negative (سلبی)
3	It's just okay, nothing special to write home ...	Neutral (محايد)

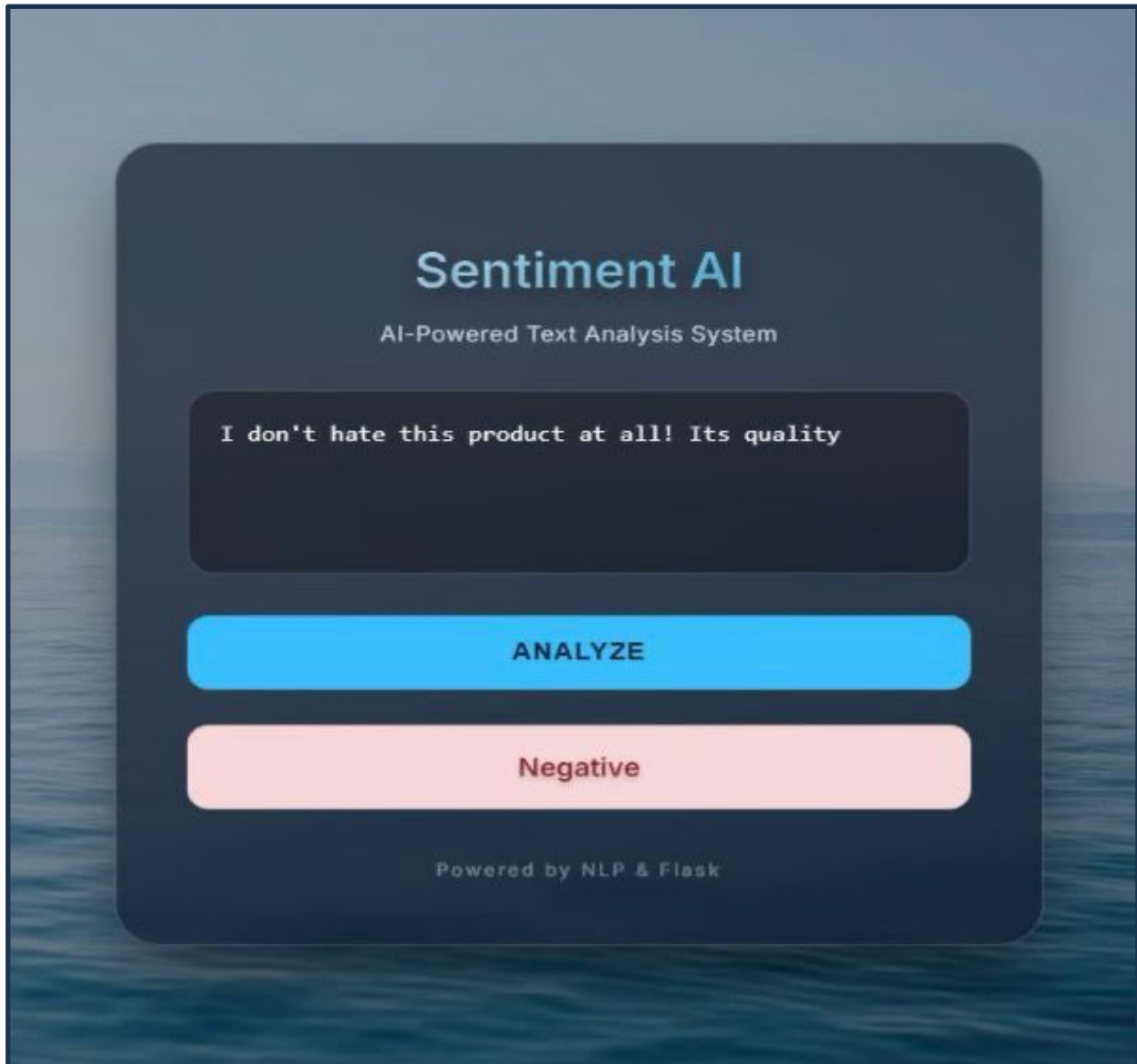
Test Sentence: 'The user interface is very confusing and hard to navigate.'

Model Prediction: Negative (سلبی)

Entity Relationship Diagram (ERD)



User Interface



Sentiment AI

AI-Powered Text Analysis System

The delivery was fast, and i love the item

ANALYZE

Positive

Powered by NLP & Flask

Sentiment AI

AI-Powered Text Analysis System

It's just okay, nothing special to write home

ANALYZE

Neutral

Powered by NLP & Flask

System Interface Implementation

A web-based interface was developed to facilitate user interaction with the sentiment analysis model. The system focuses on the following aspects:

- Backend Integration: Flask framework was used to connect the trained sentiment classification model with the web interface
- Data Input & Display: A text input area allows users to enter text, while the system displays the sentiment result (Positive, Neutral, Negative)
- State Preservation: User input remains visible after analysis to allow easy review or modification
- Local Deployment: Ngrok was used to expose the locally hosted Flask application via a public UR

System Access Steps

To access and use the system, the following steps are followed:

- The Flask server is executed in the Google Colab environment.
- A public Ngrok URL is generated automatically.
- The generated URL is accessed using any standard web browser.
- The user enters text into the input field and clicks the “**Analyze**” button to receive the sentiment classification result

Conclusion

This project presented the development of a sentiment analysis system capable of classifying text into positive, neutral, and negative sentiments using Natural Language Processing techniques

The system utilized TF-IDF for feature extraction and Logistic Regression for classification, achieving a satisfactory accuracy of approximately 80%. The results demonstrate that traditional machine learning approaches can perform effectively for text sentiment analysis tasks

In addition, a simple web-based interface was implemented using Flask, allowing users to interact with the model easily. The system can be further improved by using advanced deep learning or transformer-based models and by expanding the dataset to enhance performance, especially for neutral sentiment classification